

LAPORAN PRAKTIKUM PEKAN 5



MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 5 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Single Linked List*.

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar struktur data serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 05 Mei 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB 1	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB 2	2
PEMBAHASAN	2
2.1 <i>Single Linked List</i>	2
2.2 Class NodeSLL	2
2.3 Class TambahSLL	3
2.4 Class PencarianSLL	8
2.5 Class HapusSLL	10
2.6 Tugas Praktikum Pekan 5	13
BAB 3	28
KESIMPULAN	28
DAFTAR PUSTAKA	29
LAMPIRAN	30

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu komponen penting dalam ilmu komputer yang berfungsi untuk mengorganisasi dan menyimpan data agar digunakan secara efisien. Dalam perkembangan teknologi informasi yang semakin pesat, kebutuhan atas pengolahan data yang efektif dan fleksibel menjadi sangat penting. Salah satu jenis struktur data yang banyak digunakan adalah struktur data dinamis, yaitu struktur data yang ukuran dan kapasitasnya dapat berubah sesuai kebutuhan.

Single Linked List (SLL) merupakan salah satu bentuk struktur data yang dinamis yang terdiri dari sekumpulan node yang saling terhubung melalui pointer atau referensi. Berbeda dengan *array* yang memiliki ukuran tetap, *linked list* memungkinkan penambahan dan penghapusan elemen dilakukan dengan lebih fleksibel tanpa harus menggeser data lainnya. Setiap node pada SLL biasanya terdiri dari dua bagian, yaitu data dan pointer yang menunjuk ke node berikutnya.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar struktur data *Single Linked List*.
2. Mengetahui cara kerja node dan pointer dalam *Single Linked List*.
3. Mengetahui dan memahami operasi dasar pada *Single Linked List*.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai konsep dasar struktur data, khususnya *Single Linked List*.
2. Meningkatkan kemampuan pemrograman dalam bahasa Java.
3. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2

PEMBAHASAN

2.1 *Single Linked List*

Single Linked List (SLL) merupakan salah satu bentuk struktur data dinamis yang banyak digunakan dalam pemrograman untuk mengelola data secara fleksibel. Struktur data ini terdiri dari rangkaian node yang saling terhubung, di mana setiap node memiliki dua komponen utama, yaitu data dan pointer yang menunjuk ke node berikutnya. Berbeda dengan struktur data *array* yang memiliki ukuran tetap, SLL memungkinkan penambahan dan penghapusan elemen dilakukan tanpa harus menggeser data lain. Hal ini menjadikan SLL lebih efisien dalam kondisi tertentu, terutama ketika data sering mengalami perubahan. Selain itu, keberadaan pointer sebagai penghubung antar node membuat struktur data ini tidak bergantung pada lokasi memori yang berurutan.

Dalam praktikum yang dilakukan, implementasi *Single Linked List* difokuskan pada pemahaman cara kerja node serta manipulasi data melalui berbagai operasi dasar. Operasi tersebut meliputi penambahan node di awal, tengah, maupun akhir list, penghapusan node tertentu, serta penelusuran seluruh isi list (traversal). Proses traversal dilakukan dengan mengikuti pointer dari node pertama hingga mencapai node terakhir yang bernilai NULL. Dari sini dapat dipahami bahwa pengelolaan pointer menjadi aspek penting dalam menjaga keterhubungan antar node agar struktur list tetap konsisten dan tidak terjadi kehilangan data.

Meskipun memiliki keunggulan dalam fleksibilitas dan efisiensi manipulasi data, *Single Linked List* memiliki beberapa keterbatasan. Salah satunya adalah akses data yang bersifat sekuensial, sehingga untuk menemukan suatu elemen diperlukan proses penelusuran dari awal hingga posisi yang diinginkan. Hal ini menyebabkan waktu akses menjadi lebih lama dibandingkan *array* yang dapat mengakses data secara langsung melalui indeks. Selain itu, karena hanya memiliki satu arah pointer, SLL tidak memungkinkan traversal secara mundur. Oleh karena itu, pemilihan struktur data ini harus disesuaikan dengan kebutuhan program. Pemahaman yang baik terhadap konsep SLL akan membantu dalam pengembangan struktur data lain.

2.2 Class NodeSLL

Berikut kode program Java mengenai class NodeSLL :

```

public class NodeSLL_2511532027 {
    // node bagian data
    int data_2027;
    // Pointer ke node berikutnya
    NodeSLL_2511532027 next_2027;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532027(int data_2027)
    {
        this.data_2027 = data_2027;
        this.next_2027 = null;
    }
}

```

Program di atas merupakan implementasi dari sebuah kelas bernama `NodeSLL_2511532027` yang digunakan sebagai elemen dasar dalam struktur data *Single Linked List*. Kelas ini berfungsi untuk merepresentasikan satu buah node yang menjadi bagian dari rangkaian *linked list*. Di dalam kelas tersebut terdapat dua atribut utama, yaitu `data_2027` dan `next_2027`. Variabel `data_2027` bertipe integer yang digunakan untuk menyimpan nilai atau informasi yang dimiliki oleh node, sedangkan `next_2027` merupakan sebuah referensi atau pointer yang menunjuk ke node berikutnya dalam *linked list*. Dengan adanya pointer ini, setiap node dapat saling terhubung membentuk suatu struktur data yang berantai.

Selain itu, pada kelas ini juga terdapat sebuah konstruktor yang berfungsi untuk menginisialisasi objek node saat pertama kali dibuat. Konstruktor `NodeSLL_2511532027(int data_2027)` menerima satu parameter berupa nilai data yang kemudian disimpan ke dalam atribut `data_2027` menggunakan *keyword* `this` untuk membedakan antara variabel lokal dan atribut kelas. Pada saat yang sama, atribut `next_2027` diatur bernilai `null`, yang menandakan bahwa node yang baru dibuat belum terhubung dengan node lain atau masih menjadi node terakhir dalam *linked list*.

2.3 Class TambahSLL

Berikut kode program Java mengenai class `TambahSLL` :

```

public class TambahSLL_2511532027 {
    public static NodeSLL_2511532027
    insertAtFront_2027(NodeSLL_2511532027 head_2027, int value_2027) {

```

```

        NodeSLL_2511532027 new_node_2027 = new
NodeSLL_2511532027(value_2027);
        new_node_2027.next_2027 = head_2027;
        return new_node_2027;
    }
    // fungsi menambahkan node di akhir SSL
    public static NodeSLL_2511532027
insertAtEnd_2027(NodeSLL_2511532027 head_2027, int value_2027) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511532027 newNode_2027 = new
NodeSLL_2511532027(value_2027);
        // jika list kosong maka node jadi head
        if (head_2027 == null) {
            return newNode_2027;
        }
        // simpan head ke variabel sementara
        NodeSLL_2511532027 last_2027 = head_2027;
        // telusuri ke node akhir
        while (last_2027.next_2027 != null) {
            last_2027 = last_2027.next_2027;
        }
        // ubah pointer
        last_2027.next_2027 = newNode_2027;
        return head_2027;
    }
    static NodeSLL_2511532027 GetNode_2027(int data_2027) {
        return new NodeSLL_2511532027(data_2027);
    }
    static NodeSLL_2511532027 insertPos_2027(NodeSLL_2511532027
headNode_2027, int position_2027, int value_2027) {
        NodeSLL_2511532027 head_2027 = headNode_2027;
        if (position_2027 < 1)
            System.out.print("Invalid position");
        if (position_2027 == 1) {
            NodeSLL_2511532027 new_node_2027 = new
NodeSLL_2511532027(value_2027);
            new_node_2027.next_2027 = head_2027;
            return new_node_2027;
        }
    }

```

```

        } else {
            while (position_2027-- != 0) {
                if (position_2027 == 1) {
                    NodeSLL_2511532027 newNode_2027
= GetNode_2027(value_2027);

                    newNode_2027.next_2027 =
headNode_2027.next_2027;

                    headNode_2027.next_2027 =
newNode_2027;

                    break;
                }
                headNode_2027 =
headNode_2027.next_2027;
            }
            if (position_2027 != 1)
                System.out.print("Posisi di luar
jangkauan"); }

            return head_2027; }

            public static void
printList_2027(NodeSLL_2511532027 head_2027) {
                NodeSLL_2511532027 curr_2027 =
head_2027;

                while (curr_2027.next_2027 !=
null) {

                    System.out.print(curr_2027.data_2027+"-->");

                    curr_2027 =
curr_2027.next_2027;

                }
                if (curr_2027.next_2027==null) {

                    System.out.print(curr_2027.data_2027); }

                    System.out.println();

                }

            public static void main (String[] args)
{

                // buat linked list 2->3->5->6

```

```

NodeSLL_2511532027 head_2027 =
new NodeSLL_2511532027(2);
head_2027.next_2027 = new
NodeSLL_2511532027(3);
head_2027.next_2027.next_2027 =
new NodeSLL_2511532027(5);

head_2027.next_2027.next_2027.next_2027 = new
NodeSLL_2511532027(6);

// cetak list asli
System.out.print("Senarai
berantai awal: ");

printList_2027(head_2027);
// tambahkan node baru di depan
System.out.print("tambah 1
simpul di depan: ");

int data_2027 = 1;
head_2027 =
insertAtFront_2027(head_2027, data_2027);

// cetak update list
printList_2027(head_2027);
// tambahkan node baru di
belakang

System.out.print("tambah 1
simpul di belakang: ");

int data2_2027 = 7;
head_2027 =
insertAtEnd_2027(head_2027, data2_2027);

// cetak update list
printList_2027(head_2027);
System.out.print("tambah 1
simpul ke data 4: ");

int data3_2027 = 4;
int pos_2027 = 4;
head_2027 =
insertPos_2027(head_2027, pos_2027, data3_2027);

// cetak update list
printList_2027(head_2027);

```

```
}  
}
```

Program di atas merupakan implementasi operasi dasar pada struktur data *Single Linked List* (SLL) yang berfungsi untuk menambahkan node pada berbagai posisi serta menampilkan isi list. Kelas `TambahSLL_2511532027` menyediakan beberapa method statis yang bekerja dengan memanfaatkan kelas node yang telah dibuat sebelumnya. Method `insertAtFront_2027` digunakan untuk menambahkan node di bagian depan list, yaitu dengan membuat node baru lalu mengarahkan pointer next dari node baru tersebut ke node head yang lama. Setelah itu, node baru dikembalikan sebagai head yang baru. Cara ini sangat efisien karena tidak memerlukan penelusuran list, cukup dengan memanipulasi pointer di awal.

Selanjutnya, method `insertAtEnd_2027` digunakan untuk menambahkan node di bagian akhir list. Pada method ini, program terlebih dahulu membuat node baru, kemudian melakukan pengecekan apakah list kosong atau tidak. Jika kosong, maka node baru langsung menjadi head. Namun jika tidak kosong, program akan menelusuri list dari head hingga menemukan node terakhir (node yang pointer next nya bernilai null). Setelah itu, pointer dari node terakhir diubah agar menunjuk ke node baru. Proses ini membutuhkan traversal sehingga lebih lama dibandingkan penambahan di depan, namun tetap efisien karena tidak memerlukan penggeseran data seperti pada *array*.

Method `insertPos_2027` digunakan untuk menambahkan node pada posisi tertentu dalam list. Jika posisi yang dimasukkan adalah 1, maka prosesnya sama seperti insert di depan. Namun jika posisi lebih dari 1, program akan menelusuri node satu per satu hingga mencapai posisi yang diinginkan. Setelah posisi ditemukan, node baru disisipkan dengan cara mengatur pointer agar node sebelumnya menunjuk ke node baru, dan node baru menunjukkan ke node berikutnya. Method ini juga dilengkapi dengan validasi sederhana untuk menangani posisi yang tidak valid atau di luar jangkauan. Selain itu, terdapat method `getNode_2027` yang berfungsi sebagai pembantu untuk membuat node baru dengan nilai tertentu.

Untuk menampilkan isi *linked list*, digunakan method `printList_2027` yang melakukan traversal dari node head hingga node terakhir. Setiap data pada node akan dicetak secara berurutan dengan format panah, sehingga menggambarkan

hubungan antar node. Pada bagian main, program mendemonstrasikan penggunaan semua method tersebut dengan membuat *linked list* awal berisi data 2, 3, 5, dan 6, kemudian menambahkan node di depan (nilai 1), di belakang (nilai 7), serata di posisi tertentu (nilai 4 pada posisi ke-4). Setiap perubahan pada list langsung ditampilkan, sehingga memudahkan dalam memahami bagaimana struktur *linked list* berubah setelah dilakukan operasi penambahan.

2.4 Class PencarianSLL

Berikut kode program Java mengenai class PencarianSLL :

```
public class PencarianSLL_2511532027 {
    static boolean searchKey_2027(NodeSLL_2511532027 head_2027, int
key_2027) {
        NodeSLL_2511532027 curr_2027 = head_2027;
        while (curr_2027 != null) {
            if (curr_2027.data_2027 == key_2027)
                return true;
            curr_2027 = curr_2027.next_2027; }
        return false; }
    public static void traversal_2027(NodeSLL_2511532027
head_2027) {
        // mulai dari head
        NodeSLL_2511532027 curr_2027 = head_2027;
        // telusuri sampai pointer full
        while (curr_2027 != null) {
            System.out.print(" " + curr_2027.data_2027);
            curr_2027 = curr_2027.next_2027; }
        System.out.println(); }
    public static void main(String[] args) {
        NodeSLL_2511532027 head_2027 = new
NodeSLL_2511532027(14);
        head_2027.next_2027 = new NodeSLL_2511532027(21);
        head_2027.next_2027.next_2027 = new
NodeSLL_2511532027(13);
        head_2027.next_2027.next_2027.next_2027 = new
NodeSLL_2511532027(30);
        head_2027.next_2027.next_2027.next_2027.next_2027 =
new NodeSLL_2511532027(10);
        System.out.print("Penelusuran SLL : ");
```

```

        traversal_2027(head_2027);
        // data yang akan dicari
        int key_2027 = 30;
        System.out.print("cari data " +key_2027+ " = ");
        if (searchKey_2027(head_2027, key_2027)) {
            System.out.println("ketemu");
        }else {
            System.out.println("tidak ada");
        }
    }
}

```

Program di atas merupakan implementasi operasi pencarian (*searching*) dan penelusuran (*traversal*) pada struktur data *Single Linked List* (SLL). Kelas PencarianSLL_2511532027 menyediakan method untuk mencari suatu nilai dalam *linked list* serat menampilkan seluruh isi list. Method utama yang digunakan untuk pencarian adalah `searchKey_2027`, yang bekerja dengan cara menelusuri setiap node mulai dari head hingga node terakhir. Pada setiap node, nilai data akan dibandingkan dengan nilai yang dicari (*key*). Jika ditemukan kecocokan, method akan langsung mengembalikan nilai `true` sebagai tanda bahwa data tersebut ada di dalam list. Namun, jika seluruh node telah ditelusuri dan tidak ditemukan nilai yang sesuai, maka method akan mengembalikan `false`.

Selain itu, terdapat method `traversal_2027` yang berfungsi untuk menampilkan seluruh isi *linked list*. Method ini bekerja dengan cara mengakses node satu per satu mulai dari head hingga pointer bernilai null. Setiap data yang terdapat pada node akan dicetak ke layar secara berurutan, sehingga pengguna dapat melihat isi dari *linked list* secara lengkap. Proses traversal ini merupakan dasar dari banyak operasi pada *linked list*, karena hampir semua manipulasi data memerlukan penelusuran terlebih dahulu.

Pada bagian main, program mendemostrasikan penggunaan kedua method tersebut dengan membuat sebuah *linked list* yang berisi beberapa data, yaitu 14, 21,13, 30, dan 10. Setelah itu, program menampilkan isi *linked list* menggunakan method traversal. Selanjutnya, program melakukan pencarian terhadap suatu nilai tertentu, yaitu 30, dengan memanggil method `searchKey_2027`. Hasil pencarian

kemudian ditampilkan dalam bentuk teks “ketemu” jika data ditemukan atau “tidak ada” jika data tidak ditemukan.

2.5 Class HapusSLL

Berikut kode program Java mengenai class HapusSLL :

```
public class HapusSLL_2511532027 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511532027
deleteHead_2027(NodeSLL_2511532027 head_2027) {
    // jika SLL kosong
    if (head_2027 == null)
        return null;

    // pindahkan head ke node berikutnya
    head_2027 = head_2027.next_2027;
    // Return head baru
    return head_2027; }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532027
removeLastNode_2027(NodeSLL_2511532027 head_2027) {
    // jika list kosong, return null
    if (head_2027 == null) {
        return null;
    }

    // jika list satu node, hapus node dan return null
    if (head_2027.next_2027 == null) {
        return null;
    }

    // tentukan node terakhir ke dua
    NodeSLL_2511532027 secondLast_2027 = head_2027;
    while (secondLast_2027.next_2027.next_2027 != null) {
        secondLast_2027 = secondLast_2027.next_2027;
    }

    // hapus node terakhir
    secondLast_2027.next_2027 = null;
    return head_2027; }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511532027
deleteNode_2027(NodeSLL_2511532027 head_2027, int position_2027) {
```

```

NodeSLL_2511532027 temp_2027 = head_2027;
NodeSLL_2511532027 prev_2027 = null;
// jika linked list full
if (temp_2027 == null)
    return head_2027;
// kasus 1: head dihapus
if (position_2027 == 1) {
    head_2027 = temp_2027.next_2027;
    return head_2027; }
// kasus 2: menghapus node di tengah
// telusuri ke node yang dihapus
for (int i_2027 = 1; temp_2027 != null && i_2027 <
position_2027; i_2027++) {
    prev_2027 = temp_2027;
    temp_2027 = temp_2027.next_2027; }
// jika ditemukan, hapus node
if (temp_2027 != null) {
    prev_2027.next_2027 = temp_2027.next_2027;
}else {
    System.out.println("Data tidak ada"); }
return head_2027; }
// fungsi mencetak SLL
public static void printList_2027(NodeSLL_2511532027 head_2027) {
    NodeSLL_2511532027 curr_2027 = head_2027;
    while (curr_2027.next_2027 != null) {
        System.out.print(curr_2027.data_2027+"-->");
        curr_2027 = curr_2027.next_2027; }
    if (curr_2027.next_2027==null) {
        System.out.print(curr_2027.data_2027); }
    System.out.println(); }
// kelas main
public static void main (String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511532027 head_2027 = new NodeSLL_2511532027(1);
    head_2027.next_2027 = new NodeSLL_2511532027(2);
    head_2027.next_2027.next_2027 = new NodeSLL_2511532027(3);
    head_2027.next_2027.next_2027.next_2027 = new
NodeSLL_2511532027(4);

```

```

        head_2027.next_2027.next_2027.next_2027.next_2027 = new
NodeSLL_2511532027(5);
        head_2027.next_2027.next_2027.next_2027.next_2027.next_2027
= new NodeSLL_2511532027(6);
        //cetak list awal
        System.out.println("list awal: ");
        printList_2027(head_2027);
        // hapus head
        head_2027 = deleteHead_2027(head_2027);
        System.out.println("List setelah head di hapus: ");
        printList_2027(head_2027);
        // hapus node terakhir
        head_2027 = removeLastNode_2027(head_2027);
        System.out.println("List setelah simpul terakhir di hapus:
");

        printList_2027(head_2027);
        // Deleting node at position 2
        int position_2027 = 2;
        head_2027 = deleteNode_2027(head_2027, position_2027);
        // Print list after deletion
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_2027(head_2027);
    }
}

```

Program di atas merupakan implementasi operasi penghapusan (*deletion*) pada struktur data *Single Linked List* (SLL). Kelas HapusSLL_2511532027 menyediakan beberapa method untuk menghapus node pada berbagai posisi, yaitu di awal (head), di akhir, dan pada posisi tertentu. Method deleteHead_2027 berfungsi untuk menghapus node pertama pada *linked list*. Proses ini dilakukan dengan cara memindahkan pointer head ke node berikutnya, sehingga node pertama secara otomatis terlepas dari list. Jika *linked list* dalam keadaan kosong (head bernilai null), maka method akan langsung mengembalikan nilai null tanpa melakukan proses apa pun.

Selanjutnya, method removeLastNode_2027 digunakan untuk menghapus node terakhir dalam *linked list*. Method ini terlebih dahulu melakukan pengecekan apakah list kosong atau hanya memiliki satu node. Jika hanya satu node, maka

setelah dihapus list akan menjadi kosong. Namun jika terdapat lebih dari satu node, program akan menelusuri hingga menemukan node sebelum node terakhir (*second last*). Setelah itu, pointer dari node tersebut diubah menjadi null sehingga node terakhir terputus dari list. Proses ini membutuhkan traversal karena tidak ada pointer langsung ke node sebelumnya dalam *Single Linked List*.

Selain itu, terdapat method `deleteNode_2027` yang digunakan untuk menghapus node pada posisi tertentu. Method ini menangani dua kondisi utama, yaitu ketika node yang dihapus berada di posisi pertama (head), dan ketika node berada di posisi selain pertama. Untuk posisi selain pertama, program akan menelusuri *linked list* menggunakan dua pointer, yaitu `temp_2027` sebagai node saat ini dan `prev_2027` sebagai node sebelumnya. Setelah node yang dituju ditemukan, pointer dari node sebelumnya akan diarahkan ke node setelah node yang dihapus, sehingga node tersebut terlepas dari list. Jika posisi yang dimasukkan tidak valid atau melebihi panjang list, maka program akan menampilkan pesan bahwa data tidak ada.

Untuk menampilkan isi *linked list*, digunakan method `printList_2027` yang mencetak data setiap node secara berurutan dengan format panah. Pada bagian main, program mendemonstrasikan seluruh operasi penghapusan dengan membuat *linked list* awal berisi data 1 hingga 6. Kemudian dilakukan penghapusan node pertama, node terakhir, serta node pada posisi tertentu, dan setiap perubahan ditampilkan ke layar.

2.6 Tugas Praktikum Pekan 5

Berikut tugas praktikum yang diberikan :

Tugas Praktikum Pekan 4: Struktur Data

Topik : Implementasi Single Linked List

Tema : Simulasi Antrian Rumah Sakit

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. Aturan Penamaan (Wajib Dipatuhi)

1. Nama Kelas : Gunakan NIM lengkap.
Contoh : Pasien_2411531005 dan RumahSakit_2411531005.
2. Nama Variabel dan Method : Setiap variabel dan method wajib menggunakan 4 digit terakhir NIM.
Contoh : String namaPasien_1005, String penyakit_1005.
3. Pointer next : Variabel pointer ke node berikutnya juga wajib diberi akhiran 4 digit NIM.
Contoh : Pasien_2411531005 next_1005.

3. Struktur Program

1. Kelas ADT Pasien (Pasien_NIMLengkap)

Atribut :

- Nama Pasien
- Keluhan / Penyakit
- Nomor Antrian (int)
- Pointer next ke node Pasien berikutnya

Method :

- Constructor untuk menginisialisasi semua atribut.
- Selektor (Getter) untuk setiap atribut.

Gambar 2.1

- Mutator (Setter) untuk setiap atribut termasuk pointer next.

2. Kelas Driver (RumahSakit_NIMLengkap)

- Daftarkan Pasien (Insert at Tail) : Menambahkan node pasien baru di akhir linked list. Jika list kosong, node baru langsung menjadi head. Gunakan sebuah variabel counter untuk menyimpan nomor antrian terakhir. Setiap kali fungsi Daftarkan Pasien dipanggil, nomor antrian akan bertambah secara otomatis (*auto-increment*).
- Panggil Pasien (Delete Head) : Menghapus node terdepan dan menampilkan data pasien yang dipanggil. Head digeser ke node berikutnya.
- Tampilkan Antrian (Display) : Menelusuri linked list dari head hingga null dan menampilkan semua data pasien beserta posisinya.
- Cari Pasien (Search) : Pencarian dilakukan berdasarkan nama pasien secara **Case-Insensitive** (tidak membedakan huruf besar dan kecil). **Tampilkan pesan jika tidak ditemukan.**
- Cek Status Antrian : Menampilkan jumlah total pasien dan informasi pasien terdepan. **Tampilkan pesan jika list kosong.**

4. Contoh Tampilan Output Program

```

=== Antrian Rumah Sakit NIM: 2411531005 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

```

5. Kriteria Penilaian

1. Ketepatan struktur dan pemahaman konsep Single Linked List (pointer next, FIFO).
2. Kesesuaian aturan penamaan kelas dan variabel.

Gambar 2.2

3. Kelengkapan dan kebenaran semua operasi yang diminta.
4. Penanganan kondisi list kosong (tidak boleh error).
5. Kerapihan kode dan pemberian komentar pada bagian logika linked list.
6. Kode tugas dicommit ke github.
7. Pastikan laporan tugas ditambahkan pada laporan praktikum dibuat dengan rapi dan penjelasan yang detail serta tampilkan screenshot outputnya.

Gambar 2.3

Berikut kode program Java dari tugas praktikum :

```

public class Pasien_2511532027 {
    private String namaPasien_2027;
    private String penyakit_2027;

```

```

private int nomorAntrian_2027;
private Pasien_2511532027 next_2027;

public Pasien_2511532027(String nama_2027, String penyakit_2027, int
nomor_2027) {
    this.namaPasien_2027 = nama_2027;
    this.penyakit_2027 = penyakit_2027;
    this.nomorAntrian_2027 = nomor_2027;
    this.next_2027 = null;
}

public String getNamaPasien_2027() {
    return namaPasien_2027;
}

public String getPenyakit_2027() {
    return penyakit_2027;
}

public int getNomorAntrian_2027() {
    return nomorAntrian_2027;
}

public Pasien_2511532027 getNext_2027() {
    return next_2027;
}

public void setNamaPasien_2027(String nama_2027) {
    this.namaPasien_2027 = nama_2027;
}

public void setPenyakit_2027(String penyakit_2027) {
    this.penyakit_2027 = penyakit_2027;
}

public void setNomorAntrian_2027(int nomor_2027) {
    this.nomorAntrian_2027 = nomor_2027;
}

```

```

    public void setNext_2027(Pasien_2511532027 next_2027) {
        this.next_2027 = next_2027;
    }
}

```

Program di atas merupakan implementasi *Abstract Data Type* (ADT) yang digunakan untuk mempresentasikan satu node dalam struktur data *Single Linked List* pada simulasi antrian rumah sakit. Setiap objek dari kelas ini menyimpan data pasien yang terdiri dari nama pasien, keluhan atau penyakit, serta nomor antrian. Selain itu, terdapat atribut `next_2027` yang berfungsi sebagai pointer atau referensi ke node berikutnya dalam *linked list*. Dengan adanya pointer ini, setiap node dapat saling terhubung secara dinamis sehingga membentuk sebuah rantai antrian yang sesuai dengan konsep *Single Linked List*.

Pada kelas ini juga terdapat konstruktor yang digunakan untuk menginisialisasi seluruh atribut ketika objek dibuat. Konstruktor menerima parameter berupa nama pasien, keluhan, dan nomor antrian, kemudian menyimpannya ke dalam atribut yang bersesuaian. Sementara itu, atribut `next_2027` diinisialisasi dengan nilai `null`, yang menandakan bahwa node tersebut belum terhubung ke node lain. Hal ini penting dalam proses pembentukan *linked list*, terutama saat node baru ditambahkan ke dalam antrian.

Selain konstruktor, kelas ini juga menyediakan method getter dan setter sebagai bentuk enkapsulasi data. Method getter digunakan untuk mengambil nilai dari setiap atribut, seperti nama pasien, penyakit, nomor antrian, dan pointer ke node berikutnya. Sedangkan method setter digunakan untuk mengubah nilai atribut tersebut, termasuk mengatur hubungan antar node melalui pointer `next_2027`. Dengan adanya geeter dan setter, pengelolaan data menjadi lebih terstruktur dan aman, karena akses langsung ke atribut dibatasi.

```

import java.util.Scanner;

public class RumahSakit_2511532027 {
    static Pasien_2511532027 head_2027 = null;
    static int counter_2027 = 0;
}

```

```

        public static void daftarPasien_2027(String nama_2027, String
penyakit_2027) {
            counter_2027++;
            Pasien_2511532027 baru_2027 = new Pasien_2511532027(nama_2027,
penyakit_2027, counter_2027);

            if (head_2027 == null) {
                head_2027 = baru_2027;
            } else {
                Pasien_2511532027 temp_2027 = head_2027;
                while (temp_2027.getNext_2027() != null) {
                    temp_2027 = temp_2027.getNext_2027();
                }
                temp_2027.setNext_2027(baru_2027);
            }

            System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
" + counter_2027);
        }

        public static void panggilPasien_2027() {
            if (head_2027 == null) {
                System.out.println("Antrian kosong!");
            } else {
                System.out.println("Memanggil Pasien:");
                System.out.println("Nama      : " +
head_2027.getNamaPasien_2027());
                System.out.println("Keluhan   : " +
head_2027.getPenyakit_2027());
                System.out.println("No Antri  : " +
head_2027.getNomorAntrian_2027());

                head_2027 = head_2027.getNext_2027();
            }
        }

        public static void tampilkanAntrian_2027() {
            if (head_2027 == null) {

```

```

        System.out.println("Antrian kosong!");
        return;
    }

    Pasien_2511532027 temp_2027 = head_2027;
    int posisi_2027 = 1;

    while (temp_2027 != null) {
        System.out.println("Posisi " + posisi_2027);
        System.out.println("Nama      : " +
temp_2027.getNamaPasien_2027());
        System.out.println("Keluhan  : " +
temp_2027.getPenyakit_2027());
        System.out.println("No Antri : " +
temp_2027.getNomorAntrian_2027());
        System.out.println("-----");

        temp_2027 = temp_2027.getNext_2027();
        posisi_2027++;
    }
}

public static void cariPasien_2027(String namaCari_2027) {
    Pasien_2511532027 temp_2027 = head_2027;
    boolean ditemukan_2027 = false;

    while (temp_2027 != null) {
        if
(temp_2027.getNamaPasien_2027().equalsIgnoreCase(namaCari_2027)) {
            System.out.println("Pasien ditemukan!");
            System.out.println("Nama      : " +
temp_2027.getNamaPasien_2027());
            System.out.println("Keluhan  : " +
temp_2027.getPenyakit_2027());
            System.out.println("No Antri : " +
temp_2027.getNomorAntrian_2027());
            ditemukan_2027 = true;
            break;
        }
    }
}

```

```

    }
    temp_2027 = temp_2027.getNext_2027();
}

if (!ditemukan_2027) {
    System.out.println("Pasien tidak ditemukan!");
}

}

public static void cekStatus_2027() {
    if (head_2027 == null) {
        System.out.println("Antrian kosong!");
        return;
    }

    int jumlah_2027 = 0;
    Pasien_2511532027 temp_2027 = head_2027;

    while (temp_2027 != null) {
        jumlah_2027++;
        temp_2027 = temp_2027.getNext_2027();
    }

    System.out.println("Jumlah pasien: " + jumlah_2027);
    System.out.println("Pasien terdepan:");
    System.out.println("Nama      : " +
head_2027.getNamaPasien_2027());
    System.out.println("Keluhan  : " +
head_2027.getPenyakit_2027());
}

public static void main(String[] args) {
    Scanner input_2027 = new Scanner(System.in);
    int pilihan_2027;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM:
2511532027 ===");

```

```

System.out.println("1. Daftarkan Pasien");
System.out.println("2. Panggil Pasien");
System.out.println("3. Tampilkan Antrian");
System.out.println("4. Cari Pasien");
System.out.println("5. Cek Status Antrian");
System.out.println("6. Keluar");
System.out.print("Pilihan: ");
pilihan_2027 = input_2027.nextInt();
input_2027.nextLine();

switch (pilihan_2027) {
    case 1:
        System.out.print("Nama Pasien : ");
        String nama_2027 = input_2027.nextLine();
        System.out.print("Keluhan      : ");
        String penyakit_2027 = input_2027.nextLine();
        daftarPasien_2027(nama_2027, penyakit_2027);
        break;

    case 2:
        panggilPasien_2027();
        break;

    case 3:
        tampilkanAntrian_2027();
        break;

    case 4:
        System.out.print("Masukkan nama pasien: ");
        String cari_2027 = input_2027.nextLine();
        cariPasien_2027(cari_2027);
        break;

    case 5:
        cekStatus_2027();
        break;

    case 6:

```

```

        System.out.println("Terima kasih!");
        break;

        default:
            System.out.println("Pilihan tidak valid!");
    }

    } while (pilihan_2027 != 6);
    input_2027.close();
}
}

```

Program di atas merupakan kelas driver yang digunakan untuk mengelola seluruh operasi pada struktur data *Single Linked List* dalam simulasi antrian rumah sakit. Pada kelas ini digunakan dua variabel statis, yaitu `head_2027` yang berperan sebagai penunjuk node pertama (awal antrian), serta `counter_2027` yang digunakan untuk menyimpan dan mengatur nomor antrian pasien secara otomatis. Dengan pendekatan ini, sistem antrian mengikuti konsep FIFO, di mana pasien yang pertama masuk akan menjadi yang pertama dilayani.

Method `daftarPasien_2027` digunakan untuk menambahkan pasien baru ke dalam antrian. Setiap kali method ini dipanggil, nilai `counter` akan bertambah sebagai nomor antrian pasien. Jika *linked list* masih kosong, maka node baru langsung menjadi head. Namun jika sudah terdapat data, program akan menelusuri hingga node terakhir menggunakan variabel sementara, kemudian menghubungkan node baru di bagian akhir. Proses ini menunjukkan implementasi penambahan data pada *Single Linked List* secara dinamis.

Method `panggilPasien_2027` berfungsi untuk menghapus node paling depan sekaligus menampilkan data pasien yang dipanggil. Jika antrian kosong, program akan menampilkan pesan bahwa tidak ada pasien. Jika tidak kosong, data pasien pada head akan ditampilkan, kemudian posisi head digeser ke node berikutnya. Mekanisme ini mencerminkan konsep antrian FIFO, di mana elemen pertama yang masuk akan dikeluarkan terlebih dahulu.

Method `tampilkanAntrian_2027` digunakan untuk menampilkan seluruh data pasien dalam antrian. Program akan melakukan traversal mulai dari head hingga node terakhir (null) menggunakan perulangan `while`. Setiap node yang

dikunjungi akan ditampilkan informasi seperti nama pasien, keluhan, nomor antrian, serat posisi dalam antrian. Jika antrian kosong, maka akan ditampilkan pesan bahwa tidak ada data. Method ini membantu dalam memvisualisasikan isi *linked list* secara keseluruhan.

Method `cariPasien_2027` berfungsi untuk melakukan pencarian data pasien berdasarkan nama. Proses pencarian dilakukan dengan menelusuri *linked list* satu per satu dan membandingkan nama pasien menggunakan `equalsIgnoreCase`, sehingga pencarian bersifat tidak sensitif terhadap huruf besar atau kecil. Jika data ditemukan, informasi pasien akan ditampilkan, sedangkan jika tidak ditemukan akan muncul pesan bahwa pasien tidak ada dalam antrian.

Method `cekStatus_2027` digunakan untuk menampilkan kondisi antrian secara umum, yaitu jumlah total pasien dan informasi pasien yang berada di posisi terdepan. Program menghitung jumlah node dengan melakukan traversal dari head hingga akhir. Jika antrian kosong, maka akan ditampilkan pesan bahwa tidak ada pasien. Method ini memberikan gambaran ringkas mengenai kondisi antrian saat itu.

Pada method `main`, program dijalankan dengan menampilkan menu interaktif menggunakan perulangan `do-while`. Pengguna dapat memilih berbagai operasi seperti menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan mengecek status antrian. Input dari pengguna dibaca menggunakan objek `Scanner`, kemudian diproses menggunakan struktur `switch-case` untuk menjalankan method yang sesuai. Perulangan akan terus berjalan hingga pengguna memilih opsi keluar, sehingga program dapat digunakan secara berulang sesuai kebutuhan.

Hasil / Output :

```
=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Nama Pasien : Andi
Keluhan      : Diare
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Nama Pasien : Budi
Keluhan      : TBC
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Nama Pasien : Cici
Keluhan      : Kanker
Pasien berhasil didaftarkan! Nomor Antrian: 3
```

Gambar 2.4

```

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi 1
Nama      : Andi
Keluhan   : Diare
No Antri  : 1
-----
Posisi 2
Nama      : Budi
Keluhan   : TBC
No Antri  : 2
-----
Posisi 3
Nama      : Cici
Keluhan   : Kanker
No Antri  : 3
-----

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien: Budi
Pasien ditemukan!
Nama      : Budi
Keluhan   : TBC
No Antri  : 2

```

Gambar 2.5

```

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi 1
Nama      : Budi
Keluhan   : TBC
No Antri  : 2
-----
Posisi 2
Nama      : Cici
Keluhan   : Kanker
No Antri  : 3
-----

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 2
Pasien terdepan:
Nama      : Budi
Keluhan   : TBC

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih!

```

Gambar 2.6

```
=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien: lala
Pasien tidak ditemukan!

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 3
Pasien terdepan:
Nama      : Andi
Keluhan   : Diare

=== Antrian Rumah Sakit NIM: 2511532027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien:
Nama      : Andi
Keluhan   : Diare
No Antri  : 1
```

Gambar 2.7

BAB 3

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Single Linked List* (SLL) merupakan struktur data dinamis yang sangat fleksibel dalam pengelolaan data, terutama dalam operasi penambahan, penghapusan, pencarian, dan penelusuran elemen. Setiap elemen dalam SLL direpresentasikan sebagai node yang saling terhubung melalui pointer, sehingga memungkinkan manipulasi data dilakukan tanpa perlu menggeser elemen lain seperti pada *array*. Melalui implementasi program, dapat dipahami bahwa operasi pada *linked list* sangat bergantung pada pengelolaan pointer, di mana kesalahan kecil dapat menyebabkan struktur data menjadi tidak valid. Dengan demikian, pemahaman konsep SLL sangat penting sebagai dasar dalam mempelajari struktur data yang lebih kompleks.

Sebagai saran, diharapkan mahasiswa untuk lebih banyak berlatih dalam mengimplementasikan berbagai variasi *linked list*, agar pemahaman terhadap struktur data menjadi lebih luas. Selain itu, perlu meningkatkan ketelitian dalam pengelolaan pointer agar terhindar dari kesalahan logika program. Penggunaan visualisasi atau *tracing* langkah demi langkah juga sangat membantu dalam memahami alur kerja *linked list*. Praktikum juga dapat dikembangkan dengan menambahkan fitur lain, sehingga kemampuan analisis dan pemrograman mahasiswa semakin terasah.

DAFTAR PUSTAKA

- [1] <https://socs.binus.ac.id/2017/03/15/single-linked-list/> [Diakses 05 Mei 2026]
- [2] <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/> [Diakses 05 Mei 2026]
- [3] <https://www.wscubetech.com/resources/dsa/singly-linked-list-data-structure> [Diakses 05 Mei 2026]
- [4] <https://www.geeksforgeeks.org/dsa/linked-list-data-structure/> [Diakses 05 Mei 2026]

LAMPIRAN

Gambar 2.1	14
Gambar 2.2	15
Gambar 2.3	15
Gambar 2.4	24
Gambar 2.5	25
Gambar 2.6	26
Gambar 2.7	27